DispatchGuard — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── data/
│   ├── sample_messages/
│   │   ├── internal.txt
│   │   ├── marketing.txt
│   │   └── transactional.txt
│   └── voice_guide.md
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── brand_reviewer.py
│   │   ├── completeness_reviewer.py
│   │   ├── safety_reviewer.py
│   │   └── supervisor.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── prompts/
│   │   └── brand.txt
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── ingest.py
│   │   ├── retriever.py
│   │   └── store.py
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py
│   │   └── prompts.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── graph.py
│   ├── message.py
```

```
│   ├── schemas.py
│   └── state.py
├── tests/
│   ├── integration/
│   │   └── test_live_review.py
│   └── unit/
│       ├── test_compile.py
│       ├── test_config.py
│       ├── test_message.py
│       ├── test_routing.py
│       └── test_safety.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

Setup files

`requirements.txt`:

```
openai
langchain-openai
langgraph
chromadb
python-dotenv
pydantic
```

`.env.example`:

```
OPENAI_API_KEY=sk-your-key-here
LOG_LEVEL=INFO
```

`.gitignore`:

```
.env
.venv/
__pycache__/
*.pyc
chroma_db/
```

`LICENSE` is the standard MIT text. Every folder under `src/` gets an empty `__init__.py`.

The voice guide (`data/voice_guide.md`)

Put at least one **specific, unusual** rule in here — something a model wouldn't invent from general knowledge. That's what proves retrieval is doing the work.

```
# data/voice_guide.md

Rule B-1: Never open a customer message with "Hey" or "Yo". Use
"Hello <name>," instead.

Rule B-4: Em dashes are banned in customer copy. Use a comma or a
period instead.

Rule B-7: Always say "teammates", never "employees" or "staff", in
outgoing copy.
```

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
```

```
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/utils/prompts.py
import os

PROMPTS_DIR = os.path.join(os.path.dirname(__file__), "..", "prompts")

def load_prompt(name):
    path = os.path.join(PROMPTS_DIR, f"{name}.txt")
    with open(path) as prompt_file:
        return prompt_file.read()
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The finding shape and the message parser

```
# src/schemas.py
# One review finding. severity is high/medium/low; category is which
# reviewer raised it; message is the human-readable issue.
from pydantic import BaseModel

class Finding(BaseModel):
    severity: str
    category: str
    message: str
```

```
# src/message.py
# Split an outgoing message into its type and body. The type is a leading
# [internal] / [transactional] / [marketing] tag; no tag means marketing.
import re

def parse_message(text):
    match = re.match(r"\s*\[(\w+)\]\s*", text)
    message_type = "marketing"
    body = text
    if match is not None:
        tag = match.group(1).lower()
        if tag in ("internal", "transactional", "marketing"):
            message_type = tag
            body = text[match.end():]
    return {"type": message_type, "body": body}
```

The Safety reviewer (deterministic)

Story: a deterministic scanner is the trustworthy core — it looks at the message body for leaked PII, internal markers, and legally risky claims, and returns `Finding`s. **If not:** a purely prompted reviewer is non-deterministic and hard to pin down on a clean message.

```
# src/agents/safety_reviewer.py
# Scan the message body for leaked PII, internal markers, and risky claims.
# Deterministic, so it's testable; a scanner tested only on bad input would
# flag good copy, so test a clean message too.
import re

from src.schemas import Finding

PII_PATTERNS = [
    (r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b",
     "leaked email address"),
```

```
    (r"\b(?:\d[ -]?){13,16}\b", "possible card number"),
]
MARKER_PATTERNS = [
    (r"(?i)\b(internal|confidential|do not share)\b",
     "internal marker left in outgoing text"),
]
RISKY_PATTERNS = [
    (r"(?i)\b(guaranteed|risk-free|100% safe|cure)\b",
     "legally risky claim"),
    (r"(?i)\bi (guarantee|promise)\b", "unauthorized promise"),
]

def run_safety(message):
    findings = []
    body = message["body"]
    for pattern, label in PII_PATTERNS:
        if re.search(pattern, body):
            findings.append(Finding(
                severity="high", category="safety", message=label,
            ))
    for pattern, label in MARKER_PATTERNS:
        if re.search(pattern, body):
            findings.append(Finding(
                severity="high", category="safety", message=label,
            ))
    for pattern, label in RISKY_PATTERNS:
        if re.search(pattern, body):
            findings.append(Finding(
                severity="medium", category="safety", message=label,
            ))
    return findings
```

The RAG layer (over the voice guide)

```
# src/rag/store.py
import chromadb

COLLECTION_NAME = "voice_guide"

def get_collection(embedding_function=None):
    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
```

```
    if embedding_function is None:
        return client.create_collection(COLLECTION_NAME)
    return client.create_collection(
        COLLECTION_NAME, embedding_function=embedding_function
    )
```

```
# src/rag/ingest.py
# Chunk the voice guide into individual rules (one per paragraph) and embed
# them, so a message can retrieve the specific rules it might break.

VOICE_GUIDE_PATH = "data/voice_guide.md"

def load_rules(path=VOICE_GUIDE_PATH):
    with open(path) as guide_file:
        text = guide_file.read()
    rules = []
    for block in text.split("\n\n"):
        cleaned = block.strip()
        if cleaned.startswith("Rule"):
            rules.append(cleaned)
    return rules

def build_voice_store(collection, path=VOICE_GUIDE_PATH):
    rules = load_rules(path)
    ids = []
    for index in range(len(rules)):
        ids.append(f"rule-{index}")
    collection.add(ids=ids, documents=rules)
    return collection
```

```
# src/rag/retriever.py
def retrieve_rules(collection, query, k=3):
    found = collection.query(query_texts=[query], n_results=k)
    return found["documents"][0]
```

The Brand reviewer (RAG + LLM)

Story: retrieve the relevant voice rules for the message, then ask the model which lines break which rule — citing the **retrieved** rule, not general knowledge. **If not:** the reviewer would judge against rules it imagined.

```
# src/prompts/brand.txt
You check an outgoing customer message against the team's voice rules.
```

```
ONLY use the rules provided - never invent a rule. For each clear
violation, return a finding that quotes the rule it breaks. If there are
no violations, return an empty list.

Voice rules (the only rules that exist):
{rules}

The message:
{body}
```

```
# src/agents/brand_reviewer.py
# The Brand reviewer: retrieve the relevant voice rules for the message,
# then ask the model which lines break which rule - citing the retrieved
# rule, not general knowledge.
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel

from src.rag.retriever import retrieve_rules
from src.schemas import Finding
from src.utils.prompts import load_prompt

class BrandFindings(BaseModel):
    findings: list

class _BrandItem(BaseModel):
    line: str
    rule: str

def run_brand(model, collection, message):
    body = message["body"]
    if not body.strip():
        return []
    rules = retrieve_rules(collection, body)
    prompt = ChatPromptTemplate.from_template(load_prompt("brand"))
    chain = prompt | model.with_structured_output(BrandFindings)
    result = chain.invoke({"rules": "\n".join(rules), "body": body})
    out = []
    for item in result.findings:
        out.append(Finding(
            severity="low", category="brand", message=str(item),
        ))
    return out
```

(The model returns a list of violations under `findings`; each becomes a `Finding`. The `rules` came from retrieval, so a violation can only cite a rule that's actually in your guide.)

The Completeness reviewer (heuristic)

```
# src/agents/completeness_reviewer.py
# Flag a marketing promo with no end date, and any blast with no
# unsubscribe line. Deterministic heuristics - cheap and testable.
import re

from src.schemas import Finding

END_DATE = r"(?i)(ends|until|through|expires)"

def run_completeness(message):
    findings = []
    body = message["body"]
    lower = body.lower()
    mentions_promo = False
    for word in ("sale", "% off", "discount", "promo", "offer"):
        if word in lower:
            mentions_promo = True
    if mentions_promo and not re.search(END_DATE, body):
        findings.append(Finding(
            severity="medium", category="completeness",
            message="promo with no end date",
        ))
    if "unsubscribe" not in lower:
        findings.append(Finding(
            severity="medium", category="completeness",
            message="no unsubscribe line",
        ))
    return findings
```

The Supervisor: routing and compiling

```
# src/agents/supervisor.py
# choose_reviewers: route by message type. decide_next: the next reviewer
# that still needs to run, else compile. compile_review: merge findings
# into one de-duplicated, severity-sorted verdict. The Supervisor never
# reviews itself -- it only routes and compiles.
from langgraph.types import Command

from src.utils.logger import get_logger

SEVERITY_ORDER = {"high": 0, "medium": 1, "low": 2}
```

```
def choose_reviewers(message):
    reviewers = ["safety"]
    message_type = message["type"]
    if message_type == "internal":
        return reviewers
    if message_type == "marketing":
        reviewers.append("brand")
        reviewers.append("completeness")
    if message_type == "transactional":
        reviewers.append("completeness")
    return reviewers

def decide_next(state):
    for reviewer in state["reviewers"]:
        if reviewer not in state["done"]:
            return reviewer
    return "compile"

def compile_review(findings):
    seen = set()
    unique = []
    for finding in findings:
        key = (finding.category, finding.message)
        if key not in seen:
            seen.add(key)
            unique.append(finding)
    unique.sort(key=lambda f: SEVERITY_ORDER.get(f.severity, 3))
    if not unique:
        return "DispatchGuard: cleared to send."
    lines = ["DispatchGuard review:"]
    for finding in unique:
        lines.append(f"  [{finding.severity.upper()}] "
                     f"({finding.category}) {finding.message}")
    return "\n".join(lines)

def supervisor(state):
    logger = get_logger(__name__)
    target = decide_next(state)
    logger.info(f"supervisor -> {target}")
    return Command(goto=target,
                   update={"routing_log": state["routing_log"] + [target]})
```

The state and the graph

```
# src/state.py
from typing import TypedDict

from src.agents.supervisor import choose_reviewers
from src.message import parse_message

class ReviewState(TypedDict):
    raw: str
    message: dict
    reviewers: list
    done: list
    findings: list
    review: str
    routing_log: list

def start_state(raw):
    message = parse_message(raw)
    return {
        "raw": raw,
        "message": message,
        "reviewers": choose_reviewers(message),
        "done": [],
        "findings": [],
        "review": "",
        "routing_log": [],
    }
```

```
# src/graph.py
# START -> supervisor; the supervisor routes to each pending reviewer
# (which runs and returns to the supervisor), then to compile -> END.
# The reviewers that run depend on choose_reviewers(message), so an
# internal memo never calls Brand or Completeness.
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command

from src.agents.brand_reviewer import run_brand
from src.agents.completeness_reviewer import run_completeness
from src.agents.safety_reviewer import run_safety
from src.agents.supervisor import compile_review, supervisor
from src.state import ReviewState

def build_graph(model, voice_collection):
    def safety_node(state):
        findings = run_safety(state["message"])
        return Command(goto="supervisor", update={
            "findings": state["findings"] + findings,
```

```
            "done": state["done"] + ["safety"],
        })

    def brand_node(state):
        findings = run_brand(model, voice_collection, state["message"])
        return Command(goto="supervisor", update={
            "findings": state["findings"] + findings,
            "done": state["done"] + ["brand"],
        })

    def completeness_node(state):
        findings = run_completeness(state["message"])
        return Command(goto="supervisor", update={
            "findings": state["findings"] + findings,
            "done": state["done"] + ["completeness"],
        })

    def compile_node(state):
        return Command(goto=END, update={
            "review": compile_review(state["findings"])})

    graph = StateGraph(ReviewState)
    graph.add_node("supervisor", supervisor)
    graph.add_node("safety", safety_node)
    graph.add_node("brand", brand_node)
    graph.add_node("completeness", completeness_node)
    graph.add_node("compile", compile_node)
    graph.add_edge(START, "supervisor")
    return graph.compile()
```

main.py

```
# main.py
from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.ingest import build_voice_store
from src.rag.store import get_collection
from src.state import start_state

MESSAGE = (
    "[marketing] Hey! Our guaranteed sale is here - "
    "email me at bob@acme.com for 50% off."
)
```

```
def main():
    config = load_config()
    model = get_model(config)
    collection = build_voice_store(get_collection())
    app = build_graph(model, collection)
    final = app.invoke(start_state(MESSAGE))
    print("Routing:", final["routing_log"])
    print(final["review"])

if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "DISPATCHGUARD_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
```

```
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_message.py
# Run from the repo root: python -m tests.unit.test_message
from src.message import parse_message

def test_tagged():
    message = parse_message("[internal] secret plan")
    result = "OK"
    if message["type"] != "internal":
        result = "WRONG"
    if message["body"] != "secret plan":
        result = "WRONG"
    print(f"case 1 (tagged type + body) -> {result}")

def test_default_marketing():
    message = parse_message("Big summer sale!")
    result = "OK"
    if message["type"] != "marketing":
        result = "WRONG"
    print(f"case 2 (no tag -> marketing) -> {result}")

if __name__ == "__main__":
    test_tagged()
    test_default_marketing()
```

```
# tests/unit/test_safety.py
# Run from the repo root: python -m tests.unit.test_safety
from src.agents.safety_reviewer import run_safety
from src.message import parse_message

DIRTY = parse_message(
    "[marketing] guaranteed results - email bob@acme.com now"
)
CLEAN = parse_message("[marketing] Thanks for being a customer this year.")

def test_flags_pii_and_claim():
    findings = run_safety(DIRTY)
```

```
    text = " ".join(f.message for f in findings)
    result = "OK"
    if "email" not in text or "risky" not in text:
        result = "WRONG"
    print(f"case 1 (email + claim flagged) -> {result}")

def test_clean_message_no_findings():
    findings = run_safety(CLEAN)
    result = "OK"
    if findings != []:
        result = "WRONG"
    print(f"case 2 (clean message -> none) -> {result}")

if __name__ == "__main__":
    test_flags_pii_and_claim()
    test_clean_message_no_findings()
```

```
# tests/unit/test_routing.py
# Run from the repo root: python -m tests.unit.test_routing
from src.agents.supervisor import choose_reviewers
from src.message import parse_message

def test_marketing_runs_all():
    reviewers = set(choose_reviewers(parse_message("[marketing] hi")))
    result = "OK"
    if reviewers != {"safety", "brand", "completeness"}:
        result = "WRONG"
    print(f"case 1 (marketing -> all three)   -> {result}")

def test_internal_safety_only():
    reviewers = choose_reviewers(parse_message("[internal] hi"))
    result = "OK"
    if reviewers != ["safety"]:
        result = "WRONG"
    print(f"case 2 (internal -> safety only)  -> {result}")

def test_transactional_skips_brand():
    reviewers = choose_reviewers(parse_message("[transactional] hi"))
    result = "OK"
    if "brand" in reviewers:
        result = "WRONG"
    print(f"case 3 (transactional -> no brand) -> {result}")

if __name__ == "__main__":
    test_marketing_runs_all()
```

```
    test_internal_safety_only()
    test_transactional_skips_brand()
```

```
# tests/unit/test_compile.py
# Run from the repo root: python -m tests.unit.test_compile
from src.agents.supervisor import compile_review
from src.schemas import Finding

def test_dedup_and_sort():
    findings = [
        Finding(severity="medium", category="safety", message="x"),
        Finding(severity="high", category="safety", message="y"),
        Finding(severity="medium", category="safety", message="x"),
    ]
    out = compile_review(findings)
    result = "OK"
    if out.count("(safety) x") != 1:
        result = "WRONG"
    if out.index("HIGH") > out.index("MEDIUM"):
        result = "WRONG"
    print(f"case 1 (dedup + severity sort) -> {result}")

def test_empty_is_clean():
    out = compile_review([])
    result = "OK"
    if "cleared to send" not in out:
        result = "WRONG"
    print(f"case 2 (no findings -> cleared) -> {result}")

if __name__ == "__main__":
    test_dedup_and_sort()
    test_empty_is_clean()
```

```
# tests/integration/test_live_review.py
# Real model + RAG. Run from the repo root:
#   python -m tests.integration.test_live_review
from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.ingest import build_voice_store
from src.rag.store import get_collection
from src.state import start_state

DIRTY = (
    "[marketing] Hey! Our guaranteed sale is here - "
```

```
    "email me at bob@acme.com for 50% off."
)
INTERNAL = "[internal] confidential roadmap - do not share"

def build_app():
    config = load_config()
    model = get_model(config)
    collection = build_voice_store(get_collection())
    return build_graph(model, collection)

def test_full_review():
    final = build_app().invoke(start_state(DIRTY))
    review = final["review"].lower()
    ok = "email" in review and "end date" in review and "b-1" in review
    print(f"case 1 (full review, all 3)      -> {'OK' if ok else 'WRONG'}")

def test_internal_skips_brand():
    final = build_app().invoke(start_state(INTERNAL))
    ok = "brand" not in final["routing_log"]
    print(f"case 2 (internal skips brand)    -> {'OK' if ok else 'WRONG'}")

if __name__ == "__main__":
    test_full_review()
    test_internal_skips_brand()
```

Expected unit output:

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (tagged type + body) -> OK
case 2 (no tag -> marketing) -> OK
```

```
case 1 (email + claim flagged) -> OK
case 2 (clean message -> none) -> OK
```

```
case 1 (marketing -> all three)   -> OK
case 2 (internal -> safety only)  -> OK
case 3 (transactional -> no brand) -> OK
```

```
case 1 (dedup + severity sort) -> OK
```

```
case 2 (no findings -> cleared) -> OK
```

The two tests that carry the whole project: **routing** (an internal memo never pays for a brand check) and **compile** (one de-duplicated, severity-sorted verdict, not three raw reports). Same shape as CodeGuard — proof the architecture generalizes.

The whole project, in one sentence

A Supervisor-led team of three specialist reviewers — Safety (a deterministic PII/claim scanner), Brand (grounded in your real voice guide through RAG), and Completeness — routed by message type, compiled into one de-duplicated, severity-sorted verdict. The same patterns as CodeGuard, on an entirely different problem.